LogBoost: Boost Log Anomaly Detection by Cherry-Picking Log Sequences

Min Li, Xiaoyun Li, Pengfei Chen, Yuanhao Lai, and Zibin Zheng, *Fellow, IEEE*

***Abstract*—Debugging and operating services always benefit from logs. Since logs provide rich information on events and render comprehensive execution traces, it is imperative to automatically detect faults from extensive logs through log-based anomaly detection. However, due to the ineffectiveness and complexity of log-based detection models, they have not been widely adopted in template evaluation and real-world detection. Therefore, we propose LogBoost, a lightweight framework to boost log-based anomaly detection by automatically reducing redundant log templates. Based on our proposed similarity measurement, it effectively sorts the importance of log templates and identifies templates that are ineffective in anomaly detection. In evaluation, we introduce Spark-SDA, a new dataset featuring more diverse log templates and excessively long sequences, alongside the HDFS log dataset. We further evaluate LogBoost using established log-based anomaly detection models. The results demonstrate that eliminating ineffective log templates via LogBoost improves feature efficacy while reducing computational overhead. For instance, RandomForest obtains an F1-score at 0.983 given only 500 training samples on an optimized frequency vector of the HDFS dataset. Meanwhile, the lengths of log sequences are reduced by 51% to 77%, and the prediction time of deep learning models is reduced by 55% to 81%. Our results show that LogBoost is an effective approach to accelerate log-based anomaly detection.**

***Index Terms*—Log analysis, feature optimization, anomaly detection, deep learning.**

I. Introduction

LOGGING is an important part of system and service monitoring, as it provides rich information for debugging and fault localization. Log dataset analysis often enables operators to pinpoint the root causes of failures with greater precision and insight compared to conventional metric-based methods. However, existing logs are not specifically designed for log analysis modeling, leading to ineffective analysis and the high cost of storing massive service logs. Therefore, how to reduce redundant raw logs without losing critical information has become a serious issue. Nowadays, data-driven methods enable the automatic analysis of log data, significantly improving the effectiveness of log utilization [1], [2]. Log-based anomaly detection, as a sub-field of log analysis, assists operators in locating culprit logs [3], [4], [5], [6], [7]. To preserve the integrity of the log events and chronological context, most of the existing log-based anomaly detection methods rely on log sequences rather than single log records [5], [8], [9]. A log sequence groups a series of log records by a unique identifier [10](e.g., thread ID, task ID, job ID), such as the detailed executions of a transaction or computation task, both of which are chronological log records.

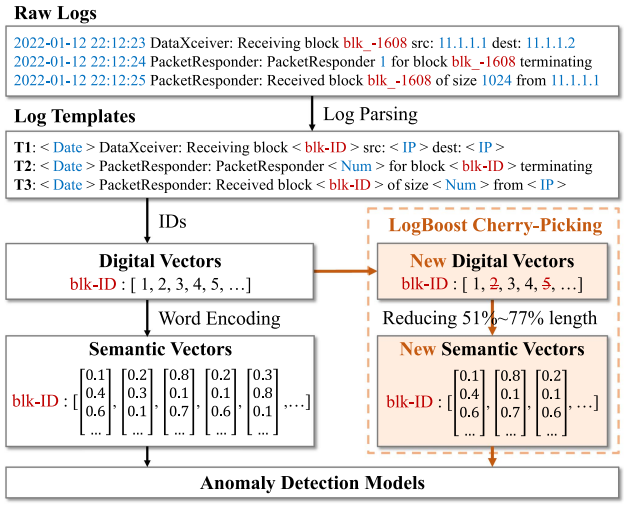


Fig. 1. Log sequences preprocessing and optimization in LogBoost.

As shown in Fig. 1, log-based anomaly detection on log sequences is usually based on two pre-processing procedures. **(i)** Log parsing converts the constant part and variables in raw log records as log templates and log variables, respectively. **(ii)** Extract features of log sequences based on parsed log templates. Generally, digital features such as sequence vectors formed by the values of log template ID, or frequency vectors that count the number of log templates are used for detection. Another popular feature is the semantic encoding. For example, Word2Vec [11] encodes the words and sentences of templates as a semantic vector. Current excellent log-based anomaly detection methods are predominantly grounded on the aforementioned preprocessing techniques [1], [12], [13]. Although they are evaluated to be effective in their experiments, they still suffer from certain limitations in real production.

Received 5 July 2024; revised 8 January 2026; accepted 8 February 2026. Date of publication 13 February 2026; date of current version 10 April 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62272495 and in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2023B1515020054. *(Corresponding author: Pengfei Chen.)*

Min Li is with the School of Systems Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China (e-mail: limin258@mail2.sysu.edu.cn).

Xiaoyun Li and Pengfei Chen are with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China (e-mail: lixy223@mail2.sysu.edu.cn; chenpf7@mail.sysu.edu.cn).

Yuanhao Lai is with the Huawei Technologies, Shenzhen 518129, China (e-mail: laiyuanhao@huawei.com).

Zibin Zheng is with the School of Software Engineering, Sun Yat-sen University, Guangzhou 510006, China (e-mail: zhzibin@mail.sysu.edu.cn).

Digital Object Identifier 10.1109/TSC.2026.3664358

Based on the research and experiments on the real-world system [14], [15], [16], we conclude three drawbacks of existing log-based anomaly detection methods. **(i)** Since the error logs intermingle with normal logs, the normal and abnormal log sequences always share common sub-sequences. It enhances the similarity and length of sequences as well as the feature dimensionality, thereby impacting the performance and efficiency of the detection models. **(ii)** Due to the variability of industry logging, it is challenging to establish a universal prior knowledge of evaluating logs. Despite most log-based anomaly detection models achieving high accuracy, they are incapable of assessing the contribution of each log template in anomaly detection. **(iii)** Semantic features allow semantically similar sentences to be closer in the hyperspace, enabling a higher generalizability of detection models. However, the higher dimensional semantic vectors (e.g., 300 dimensions generated by the FastText [17]) introduce a substantial overhead in terms of time and resource consumption.

We investigate various approaches to feature dimensionality reduction and sequence mining to address the aforementioned issues. Firstly, considering the feature dimensionality reduction, MultiDimensional Scaling (MDS) [18] and Stochastic Neighbor Embedding (SNE) [19] are widely used methods to reduce the feature dimension. But they lose the semantics of the original features after optimization, and cannot tell which and how features are selected. Furthermore, encoding semantic vectors of log events by deep neural networks is an effective method for feature optimization [20], [21], but these methods also fail to provide an assessment of the effectiveness of log templates. Secondly, separating key logs (logs that are useful for anomaly localization) and redundant logs in log sequences can significantly improve the effectiveness of features for sequence classification. Mining sequence patterns is a widely used method for extracting crucial features of sequences. Currently, mining sequence patterns for sequence classification is well established and applied, which primarily employ frequent item mining to extract sequential sequence patterns as features for classification [22], [23], [24], [25]. However, when mining frequent items in log sequences, the primary outcome is high-frequency logs with INFO and WARN levels, as well as ERROR logs that are generated frequently during system malfunctions.

In this paper, we propose LogBoost, a lightweight log sequence optimization framework, that is designed to automatically filter ineffective or redundant log templates in anomaly detection without any prior knowledge. LogBoost ranks the importance of log templates based on our novel Markov-based measurement with only simple features instead of semantic features. As shown in Fig. 1, by eliminating ineffective log templates, such as template IDs 2 and 5 are identified as ineffective templates, the length of log sequences is reduced significantly as well as the dimension of semantic vectors. Our experiments show that the maximum length of sequences in the HDFS dataset [26] is decreased by more than 70%, greatly reducing the time for model training and detection, the prediction time of deep learning models are reduced by 55% to 81%. With LogBoost optimization, lightweight models can readily achieve higher accuracy with a smaller training set. For example, RandomForest

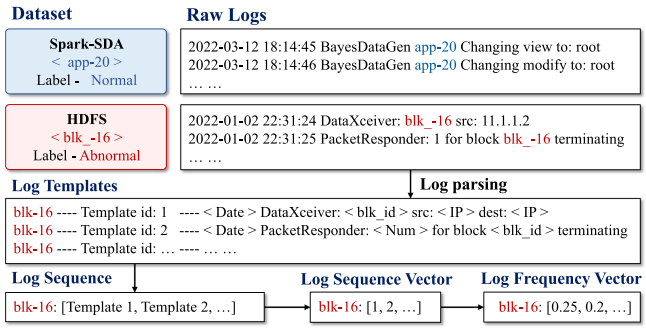


Fig. 2. Example of definitions in log preprocessing.

achieves 0.983 F1-Score given only 500 (0.087%of the total dataset) training samples on optimized frequency vectors of the HDFS dataset. Our results show the efficiency of LogBoost on Log-based anomaly detection models [27], [28], [29].

The main contributions of this paper are summarized as follows.

- We propose a novel method to measure the similarity among log sequences based on the State Transition Probability Matrix, which is used to assess and rank the efficacy of log templates more accurately.
- We propose LogBoost, a log optimization framework, that filters out ineffective or redundant log templates in log sequences for boosting anomaly detection.
- As the HDFS dataset [26] is over-simplicity and no longer representative in modern applications. Based on the Apache Spark [30], we construct a higher complex Spark-SDA log dataset with normal and anomaly labels to further evaluate to further evaluate LogBoost.
- Extensive and rigorous experiments are conducted to validate that LogBoost can effectively optimize log sequences, and boost existing anomaly detection methods.

The rest of paper is organized as follows. Section II gives the important definitions. Section III introduces our investigation and motivation. Section IV describes the methodology including an optimization example of LogBoost. The experimental results are shown in Section V, and we give a case study in Section VI and discussion in Section VII. Related works are reviewed in Section VIII. Section IX provides an overall conclusion of our work.

II. Definition

Log template: As shown in Fig. 2, the log parsing stage separates log templates and parameters "<∗>" (e.g., <Date>, <blk_id>, <IP>, <Num>). The log template is the constant part of a log message (e.g., *"DataXceiver: <*> src: <*> dest: <*>"*, *"PacketResponder: <*> for block <*> terminating"*), which is mainly designed by developers.

Log sequence: After parsing, the raw log are typically grouped into log sequences. They are listings of chronological log records that are correlated by unique identifiers in raw logs. As shown in Fig. 2, <blk_id> in the HDFS dataset correlates a series of log records (e.g., blk_-16), as well as an anomaly or normal label. Likewise, in the Spark-SDA dataset, <app-id> is the

unique identifier to correlate log sequences (e.g., app-20). As for logs lacking sequence identifiers, a fixed window is commonly employed to segment the log sequences.

Log sequence feature: During the parsing, different log templates are numbered into positive integers according to their arriving order, starting from 1, and the increment step is 1. The template set is denoted as $T = \{t_1, t_2, \ldots, t_H\}$, $t_i \in N^*$, t_i is the ID of a log template by a positive integer, H is the total number of log templates. Thus, a log sequence vector l is denoted as $l = [t_i^1, t_i^2, \ldots, t_i^L]$, $t_i^j \in T$, where L is the length of log sequence.

Log frequency feature: For each log sequence, its frequency vector f is generated based on the template frequency of sequence vector l, $f = [\frac{count(t_1)}{L}, \frac{count(t_2)}{L}, \ldots, \frac{count(t_H)}{L}]$, where $count(t_i)$ is the occurrence number of the log template t_i in sequence l, and L is the length of the sequence.

III. Motivation

A. The Evaluation of Log Template Effectiveness

Logs are primarily designed for human debugging rather than for automated data-driven anomaly detection [4], [31]. They usually contain common subsequences logs between normal and abnormal log sequences [32]. We divide the HDFS logs [26] into normal and abnormal sequences as two subsets and deduplicate them separately. Then, we apply the PrefixSpan algorithm [33] for frequent subsequence mining and extract subsequences that cover 80% of the dataset with a length exceeding 10 (Since the average of the sequence length in HDFS is 19, we opt to take half of this value). The results show that the number of frequent subsequences in the two sets is 550 and 848, with a total of 237 identical subsequences. It indicates that normal and abnormal sequences share numerous identical subsequences. Consequently, we define ineffective log templates as those present in both sequence types, which impede the identification of abnormal patterns. These templates substantially increase sequence length and similarity, thereby degrading the effectiveness and efficiency of log-based anomaly detection.

To investigate the influence of shared log templates, we removed the intersections between normal and abnormal sequences and evaluated XGBoost, DeepLog, and RobustLog. All models exhibited a reduction in accuracy. The substantial decline in XGBoost and DeepLog suggests that the ordering of specific templates is vital for their classification. Meanwhile, the resulting sparsity in contextual information constrained RobustLog's semantic reasoning, rendering it less effective at identifying failure patterns.

Existing detection methods identify abnormal patterns within log sequences or messages, they typically transform raw logs into high-dimensional features, resulting in an inability to evaluate the contribution of specific log templates during detection. For example, machine learning-based methods (e.g., XGBoost) and attention mechanisms can quantify feature contributions, but these relate to fixed positions in the feature vector rather than specific log templates. Therefore, it is difficult to automatically evaluate the contribution of log templates in anomaly detection without expert knowledge [31]. However, relying solely on template distribution may lead to the neglect of critical logs with *info* and *warn* levels or *error* logs that are numerous on certain exceptions.

B. The Similarity Measurements on Log Sequence

Currently, assessing the similarity among sequences is mainly based on the vector or set similarity, such as Euclidean distance, Cosine similarity, Pearson Correlation Coefficient and Dynamic Time Warping (DTW) [34]. Based on an extensive survey of log sequences in the public HDFS dataset [26], several key issues affect the similarity among log sequences. For the below two log sequence examples from the HDFS dataset, where each numerical ID in these sequences corresponds to a distinct log template as defined in Section II, and the entire sequence encapsulates a comprehensive operation log for a block. We consider several sequence changes that commonly occur during log engineering to evaluate existing similarity measurements.

<**blk_6986538622634403948**>: 1 2 1 1 3 4 3 4 3 4 5 5 5 11 11 11 34 34 34 10 34 34 34 34 34 34 34 34 34 34 34 34 31 31 31 15 15 15

<**blk_6035208985581955117**>: 1 1 2 1 3 4 4 3 3 4 5 5 5 11 11 11 34 34 34 34 34 34 34 34 34 34 34 34 34 34 10 31 31 31 15 15 15

Template changes: In systems employing an unique log parser, the parsed log template IDs remain invariant. However, modifications to logging statements during code development can disrupt this stability. Specifically, alterations to a log statement may lead the parser to identify it as a new template, thereby replacing the original template ID within the log sequence. This substitution alters the template ID in sequences, which consequently impacts similarity calculations based on template IDs. Consider the case where template ID 34 in the sequence changes to 40 in the example above. The numerical values in vectors directly affect the value-based distance measurements, such as Euclidean distance, Cosine similarity, Pearson Correlation Coefficient and DTW.

LCS in context: The Longest Common Subsequence (LCS) increases the context similarity, for instance, the subsequence $[t_1, t_5, t_9]$ of the sequence $[\mathbf{t_1}, t_2, \mathbf{t_5}, t_4, \mathbf{t_9}]$ and the sequence $[\mathbf{t_1}, t_3, \mathbf{t_5}, t_7, \mathbf{t_9}]$ is an LCS. Our investigation of HDFS [26] and BGL [35] log datasets reveals that LCS is prevalent in both normal and anomalous sequences. They substantially inflate similarity scores of the Cosine, Pearson, Jaccard, and DTW. Furthermore, consider removing the LCS. We eliminate the segments in the two sequence instances that share identical locations and templates (change to blk_*48: [2,1,3,4,10,34], blk_*17: [1,2,4,3,34,10]). However, the above methods, including Euclidean distance, fail to distinguish those two entirely disparate sequences based on their similarity scores.

Sequence length: It is difficult to measure the similarity of over-length and contextual correlated log sequences. The DTW algorithm retains the context of sequences, but its efficiency is greatly affected by the sequence length. Meanwhile, repetitive

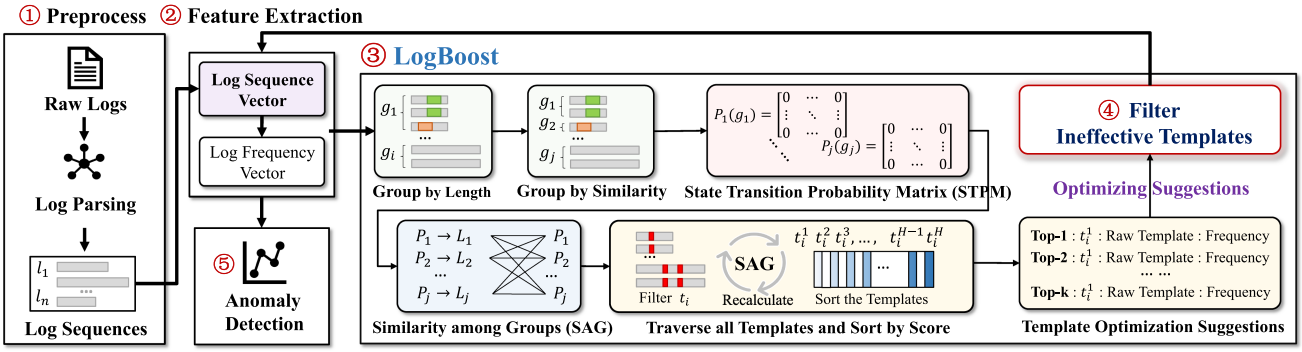


Fig. 3. Overview of LogBoost framework in log-based Anomaly Detection.

TABLE I
THE OVERHEAD OF STATE-OF-ART LOG ANALYSIS

Model	Log Features	Compute Device	Train Time	Prediction Time	Memory Required
DeepLog [27]	sequence	GPU	8.51m	41.5m	8GB
LogAnomaly [28]	sequence & count	GPU	16.9m	69.4m	14GB
RobustLog [29]	semantic	GPU	27.1m	25.6s	32GB
PLELog [9]	semantic	GPU	32.4m	46.3s	32GB

• m (minute), s (second).

and cyclic subsequences, such as IDs: $\{5, 11, 34\}$ in example sequences, increase their similarity and maximum length. We evaluate the computational efficiency of DTW by replicating the example sequence above. The original sequence has a length of 38, when the sequence length is 1,140 (replicating 30 times), the time to calculate similarity between two sequence is 2.18 seconds, and when the sequence length is up to 3,040 (80 times), it costs 15.4 seconds. This results indicate that the DTW is significantly influenced by the length of sequences and struggles to process excessive-long log sequences.

C. The Overhead of Log Analysis

We validate the state-of-the-art models on the Spark-SDA dataset which has 22,110 log sequences and over 33% of sequences have lengths exceeding 550. The cost of training, prediction and resources are shown in Table I. All of the models need a GPU device for acceleration, and the complexity of the log sequences significantly impacts the memory requirement, considering the peak memory utilization during the prediction of the entire dataset. The prediction time of RobustLog and PLELog is relatively short, as it processes multiple sequences in batches, and there are only 18,708 sequences for prediction. Therefore, existing deep-learning anomaly detection models are analysis costly and hard to transfer to current applications [36].

IV. METHODOLOGY

A. The Overview of LogBoost Framework

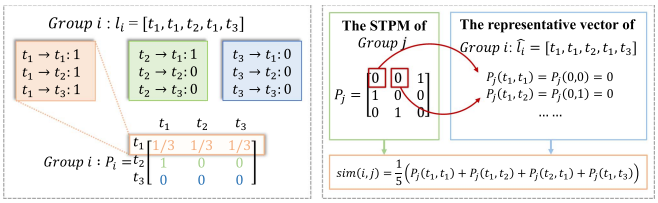
The overview of the LogBoost is shown in Fig. 3. The input of LogBoost is the log sequence vectors, and the output is the boosted dataset and log templates to be filtered out. In steps 1 and 2, we parse raw logs into templates and group log sequences by unique identifiers (e.g., blk_id) to construct log sequence vectors. In step 3, LogBoost starts up based on log sequence vectors, they are grouped by length and position similarity at first. LogBoost calculates the State Transition Probability Matrix (STPM) for each group and computes Similarity Among Groups (SAG) based on STPMs. Then, LogBoost filters out a log template in sequence vectors and recalculates new SAG′ to evaluate the deviation score of this template. After traversing all templates, LogBoost outputs top-k log templates. In step 4, those templates are directly filtered out in log sequence vectors, and log frequency vectors are regenerated with optimized log sequence vectors. Finally, in step 5, the optimized data is forwarded to the anomaly detection model for analysis. **Therefore**, LogBoost is a feature-level optimization framework before anomaly detection, it does not decrease the number of log sequences but only mitigates the negative impact of ineffective templates within them.

B. A Novel Similarity Measurement in LogBoost

Since detecting anomalous patterns in log sequences constitutes a classification task, log templates that enhance sequence similarity are counterproductive. Removing such templates improves the classification accuracy of both log sequences and anomalous patterns. However, existing standard similarity metrics (e.g., Cosine similarity) often fail on ultra-long log sequences because they primarily account for template frequency while ignoring the intrinsic execution logic. Since logs are generated by structured program execution, the order of events is a more robust signature than their count.

Considering the issues in Section III-B, and motivated by the Markov chain model for sequence data [37], we propose a novel similarity measurement based on the **S**tate **T**ransition **P**robability **M**atrix **(STPM)**, which is not affected by the numerical value while keeping their context information and can handle excessive-long sequences. The calculation process is shown in Fig. 3 at step 3, the details are presented as follows.

i) Group by Length and Similarity: Suppose a set of log sequences as $S = \{l_1, l_2, \ldots, l_N\}$, N is the total number of log sequences. Those sequence vectors are grouped by their length at first. However, only grouping by sequence length may lead to grouping different operations with the same length. Therefore,



(a) Example of Calculating STPM (b) Example of Calculating $sim(i, j)$

Fig. 4. Example of calculating STPM and similarity.

we further separate log sequences according to their differences in context position, ensuring different type of sequences are not in the same group. For example, the length of the sequence A and B are L. Based on the sequence order of A, the number of different templates in the same position of A and B is denoted as L', when $\frac{L'}{L} \geq \alpha$, $\alpha \in [0, 1]$, separating sequence B into a new group. Finally, the representative vector $\hat{l}$ for a group is constructed by templates with the highest occurrence frequency at each position.

ii) Calculate STPM: After grouping, STPM is calculated for each group, denoted as P_i, which is a $H \times H$ matrix and H is the total number of log templates in all groups. Fig. 4(a) shows a calculation example, suppose there are 3 templates $[t_1, t_2, t_3]$, and only one log sequence vector $l = [t_1, t_1, t_2, t_1, t_3]$ in group i, where log template t_1 transits to t_1, t_2, and t_3 once respectively. The first row of P_i is the State Transition Probability (STP) of the template t_1, which is the number of transitions to each possible state divided by its total transitions. Similarly, the second and third rows correspond to the STP of templates t_2 and t_3 calculated in the same way. When there is more than one sequence vector in a group, it needs to sum the state transition times in all vectors to calculate STP for each template. Therefore, STPMs of all groups are denoted as $E = \{P_1, P_2, \ldots, P_M\}$, $M \leq N$, M is the total number of groups.

iii) Similarity Among Groups: The similarity among groups is measured based on STPMs. As shown in Fig. 4(b), $\hat{l}_i$ is the representative sequence vector of group i, the STPM of group j is P_j, the similarity from group i to j is $sim(i, j)$ defined as (1). $\hat{l}_i(k)$ is the log template ID at position k $(1 \leq k \leq L-1)$ of $\hat{l}_i$ with length L. $P_j(t_m, t_n)$ is the value in STPM indicating log template t_m transits to t_n.

$$sim(i, j) = sim(\hat{l}_i, P_j) = \frac{1}{L} \sum_{k=1}^{L} P_j\left(\hat{l}_i(k), \hat{l}_i(k+1)\right). \quad (1)$$

The representative vector of a group i always encounters zero or less probability in STPMs of other groups, thus $sim(i, i) > sim(i, j)$. It means that the more similar group i and j, the greater the value of $sim(i, j)$ is. To evaluate the separation degree of all log sequences of the dataset, we define Similarity Among Groups (SAG) as (2), whose computational complexity is $O(M^2)$. To accelerate the calculation, based on the distributive property of matrix multiplication, it can be further simplified by rewriting it as (3), which has computational complexity $O(M)$.

$$SAG = \sum_{i=1}^{M} \sum_{j=1}^{M} sim(i, j), \quad (2)$$

$$SAG = \sum_{i=1}^{M} sim(i, P_{sum}), \quad P_{sum} = \sum_{j=1}^{M} P_j. \quad (3)$$

As for the key issues in IV-B, when template IDs from 34 change to 40, LogBoost is not affected and distinguishes entirely distinct sequences after removing LCS. As for efficiency, when the sequence length is 3,040, it costs only 1.79 milliseconds. Therefore, LogBoost can efficiently handle large-scale long sequences.

C. Effectiveness Evaluations of Log Templates

Conceptually, SAG quantifies the total overlap across the transition spaces of all identified groups. A high SAG value indicates that anomalous and normal sequences share highly similar transition patterns, making them difficult to distinguish. Therefore, the optimization goal of LogBoost is to minimize SAG. Decreasing SAG can increase the hyperspace distance among groups, leading to a more accurate classification. Suppose SAG_{base} is the original SAG of dataset $\mathbb{V}$, then evaluate the effectiveness of a log template t_i as follows: **(i)** Filter out log template t_i in log sequence vectors of $\mathbb{V}$, and obtain a new dataset $\mathbb{V}'$. **(ii)** Recalculate the SAG_i by $\mathbb{V}'$. **(iii)** Consider the $\Delta SAG_i = SAG_{base} - SAG_i$.

Evaluation Rules: ❶ When $\Delta SAG_i > 0$, filtering out the log template decreases the hyperspace similarity of log sequence vectors, these log templates are redundant or contained in the LCS. ❷ When $\Delta SAG_i < 0$, filtering out the log template increases the hyperspace similarity of log sequence vectors, these log templates are important ones with a strong contribution to classification. ❸ When $\Delta SAG_i = 0$, the log template does not appear in this dataset, considering that the training set may not contain the whole set.

Notably, the above method has limitations when handling log sequences comprising substantial distinct task types (e.g., the Spark-SDA dataset), as template intersections across sequences of different tasks are sparse, often resulting in massive $\Delta SAG_i = 0$. To solve this problem, if LogBoost detects that issue in more than 10% of templates during analysis, it will prompt that the dataset needs to be classified manually. Generally, it is recommended to classify based on component or task type. Therefore, we partition the Spark-SDA dataset by task type before analysis. After LogBoost traverses all log templates and calculates ΔSAG_i for each log template, LogBoost then arranges them in descending order, and outputs top-k log templates with $\Delta SAG_i > 0$ for filtering.

D. Feature Optimization

The optimization of LogBoost is based on log sequence features, as shown in Fig. 5, it shows the optimization process on HDFS log datasets. Firstly, the ΔSAG_i of templates are calculated and sorted. The analysis of the frequency distribution of template occurrences reveals that templates with higher scores do not necessarily exhibit the highest frequencies. Subsequently, these redundant templates are filtered, where l_i represents the raw log sequence, while the highlighted (red) portion represents the templates subject to filter.

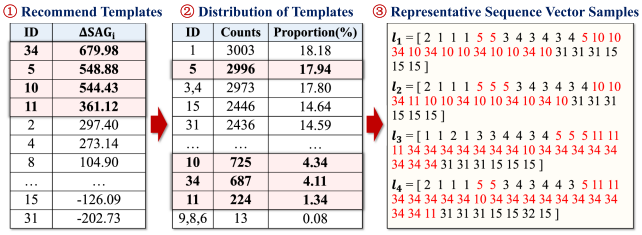


Fig. 5. Optimization example of HDFS log dataset.

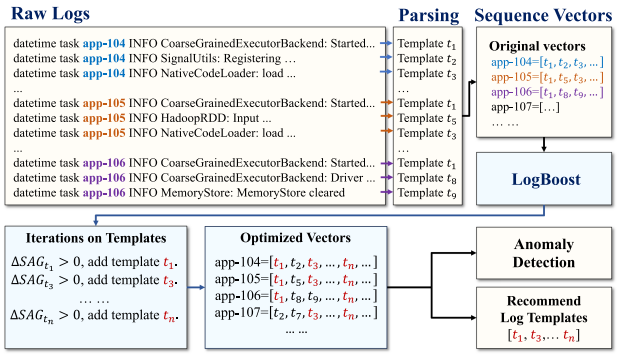


Fig. 6. Example of LogBoost optimization process.

As for the optimization of frequency and semantic features, both of which can be regenerated based on new sequences. The frequency feature can be regenerated using the new log sequence features, while semantic features are derived by mapping each log template to high-dimensional vectors.

E. Log Boosting Process

There is an optimization example of LogBoost in Fig. 6. The raw Spark logs are parsed into log templates and grouped into log sequence vectors with unique identifiers (e.g., app-id). Then LogBoost iteratively calculates ΔSAG_t for each log template and generates optimized vectors by templates with $\Delta SAG_t > 0$ for anomaly detection. Hence, LogBoost serves as a feature-level optimization framework that can be integrated into the existing anomaly detection methods as a feature engineering procedure. When the set of log templates is unchanged, it only needs to be executed once. The obtained template can then be directly used for feature optimization in subsequent anomaly detection.

V. Evaluation

Our evaluation addresses the following research questions.

- *RQ1:* How well can LogBoost improve the accuracy and performance of anomaly detection models?
- *RQ2:* How well can LogBoost optimize log dataset?
- *RQ3:* What are the impacting factors of LogBoost?

A. Dataset

Log-based anomaly detection methods commonly use the HDFS [26], BGL [35] log datasets as benchmarks. Since LogBoost is primarily designed to analyze log sequences, it requires raw data including unique identifiers to identify and delineate log sequences. We construct a more complex and labeled Spark-SDA log dataset which is similar to the HDFS dataset for evaluation. The statistical comparison of the datasets is shown in Table III, and the Spark-SDA dataset has a higher length of log sequence.

HDFS log dataset: It is a log dataset published in work [26], which is widely used in log-based anomaly detection research. The detailed raw logs are generated by running Hadoop-based map-reduce jobs on more than 200 Amazon EC2 nodes. Each line of raw logs contains a file block id (blk_id) indicating a complete log sequence of block operations, and each log sequence is labeled by experts as normal or anomaly.

Spark-SDA log dataset: We constructed it by running 29 types of standard Apache Spark [30] jobs in the standalone mode. We deploy the Spark system on a cluster consisting of one master and one slave server. Sample tasks are randomly generated using automated scripts, and Chaosblade [38] are employed for fault injection during the execution process. We collect the task execution logs over a week. The raw logs of each job are stored in separate files, and we aggregate all log files together and use the app_id in each job as a unique identifier for a complete execution sequence. Moreover, We record the task type of each sequence as well as the faults injected in the Spark-SDA dataset. We marked tasks finished successfully as normal and tasks with injected faults in the logs are marked as anomalies. The detailed task types and injected faults are shown in Table II. Spark-SDA contains 29 types of tasks and 7 types of faults since some tasks do not generate logs in testing, and we choose the types of fault injection that have obvious impacts on logs. Such as server network failures (e.g., fault injection type (2)(3)), which cause network connection error logs and retry logs. Due to the limitation of paper space, detailed generation and injection instructions are available in our artifacts.

BGL log dataset: It is generated by the Blue Gene/L supercomputer, which consists of 128 K processors and is deployed at Lawrence Livermore National Laboratory (LLNL) [35]. Each line of log is manually labeled as either anomalous or normal. However, since BGL dataset lacks a standard identifier for partitioning log sequences, we employ a fixed window size in supplementary experiments to validate LogBoost.

Based on the HDFS and Spark-SDA log datasets, we extract three types of datasets for evaluation. ❶ HDFS-A is generated by the HDFS log dataset based on DeepLog [27] using Spell [39] log parser, which converts raw logs into 29 event types (each line of log is represented by a numerical value $t_i \in [1, 29]$). Then construct log sequence vectors by the numerical ID of log templates. Furthermore, considering the low accuracy of log template extraction [40], the log parser of SwissLog [9] is used to generate the following new dataset (the comparison of log parser is introduced in Section V-G). ❷ HDFS-B is generated by the HDFS log dataset using the log parser of SwissLog, which generates 48 log templates, then constructs log sequence vectors (templates ID: $t_i \in [1, 48]$). ❸ SPARK is generated by the Spark-SDA log dataset using the SwissLog parser (158 log templates), and constructed to log sequence vectors (templates ID: $t_i \in [1, 158]$). Additionally, since RobustLog [29] needs semantic features, we generated the semantic features of the

TABLE II
TASK TYPES AND FAULT INJECTION DETAILS IN SPARK-SDA DATASET

Classification of Task Types					Types of Faults
ALS	Gradient	ScalaTeraSort	ScalaRepartition	RatingDataGeneration	(1) Out of disk space
SVM	ScalaSort	ScalaPageRank	PCADataGenerator	RandomForestDataGenerator	(2) Server network failure in HDFS port
RFC	ScalaSleep	ScalaWordCount	LDADataGenerator	LogisticRegressionWithLBFGS	(3) Server network failure in spark port
LDA	PCAExample	NWeightGraphX	SVDDataGenerator	LogisticRegressionDataGenerator	(4) Software failure shutdown HDFS
SVD	DenseKMeans	GraphXPageRank	NaiveBayesExample	GradientBoostedTreeDataGenerator	(5) Software failure kill spark process
NWeight	BayesDataGen	SVMDataGenerator	GaussianMixtureModel		(6) Delete HDFS data files
					(7) Corrupt HDFS data files

TABLE III
THE STATISTICS OF LOG DATASET

Datasets	HDFS	Spark-SDA	BGL
Log Lines	10.65 M	15.62 M	4.49 M
Data Size	1.47 GB	2.22 GB	705 MB
Length Range	[2, 298]	[4, 6242]	20
Average Length	19.4	741.2	20
Anomaly	16,838	1,267	20,256
Log Event	48	156	688

HDFS and Spark-SDA log datasets using the FastText algorithm [17] and TF-IDF [41]. When using HDFS-A, HDFS-B and Spark as input, each log template ID corresponds to a 300-dimensional semantic vector.

B. Anomaly Detection Models

Since log-based anomaly detection is a classification task, we choose the simple and efficient Ensemble Learning Models model for classification evaluation. To evaluate the enhancement of LogBoost on existing models, we deploy several LSTM-based deep learning models. The details are as follows.

Ensemble Learning Models: RandomForest (RF) [42] and XGBoost [43] are extensively used supervised classification models, which are composed of multiple decision trees and output binary classification probability to detect anomalies.

DeepLog [27]: An unsupervised model with classical Long Short-Term Memory (LSTM) to learn the context information of log sequences. It predicts the top-k probability of the next log template in sequences to determine abnormality, where k is a hyperparameter as an anomaly threshold.

LogAnomaly [28]: An unsupervised model with LSTM units to learn sequential and quantitative patterns. It considers the counts of different log templates as an additional feature and also detects anomalies by predicting the next log template.

RobustLog [29]: A supervised model with Bidirectional Long Short-Term Memory (Bi-LSTM) units, which utilizes semantic features derived from log sequences to detect anomalies through binary classification probability.

In the above models, RandomForest and XGBoost are set to fit binary classification and parameters remain as default, while DeepLog, LogAnomaly and RobustLog are matched to different datasets according to the configuration provided in their papers. Moreover, since the input vectors of some models need to be of equal length, we use 0 to pad vectors. It does not affect the value of log sequence vectors, since the template ID starts from 1 in our log parsing.

C. Metrics

The accuracy of anomaly detection models is evaluated by F-measure, including False Positive (FP), True Negative (TN), False Negative (FN) and True Positive (TP). Recall (R) and Precision (P) are defined as $R = \frac{TP}{TP+FN}$, $P = \frac{TP}{TP+FP}$, the F1-Score is defined as $F_{score}(\beta = 1) = \frac{2 \cdot P \cdot R}{P+R}$.

Additionally, log parsing and feature optimization are regarded as uniform preprocessing works, and the performance of models is primarily evaluated by training and prediction. To unify the time comparison criterion, the neural network models are all trained for 500 iterations, and we validated that all models achieve a stable loss at that stage.

D. Baseline Models and Experiment Setup

We select PrefixOpt, LogAssist [44], and LogCleaner [25] as baselines. While PrefixOpt identifies redundant templates via frequent item mining [45], LogAssist employs n-gram modeling and sequence hiding, and LogCleaner focuses on event reduction for anomaly detection. In our experiments, LogBoost and these baseline models optimize using log sequence vectors. Notably, log frequency vectors are restricted to ensemble learning models, as sequence models like DeepLog and LogAnomaly require integer-based template IDs.

Parameters: There are two parameters α and top_k of LogBoost, where α is a hyper-parameter used to avoid the existence of different log sequences in the group with the same length, it is related to the average length of log sequences. Generally, if a lower threshold is set in long sequences, there are more groups. We validate that α from 0.1 to 0.4 are suitable for each dataset. Moreover, top_k is set to filter out default top-k log templates, it is related to the total number of templates. We validate that the top k templates with $\Delta SAG_i > 0$ for each dataset. A detailed discussion of parameter selection is presented in the Section V-G. To ensure consistent comparison conditions, both PrefixOpt and LogAssist set the same top-k as LogBoost. In summary, ❶ HDFS-A, set $\alpha = 0.2$, $top_k = 2$; ❷ HDFS-B, set $\alpha = 0.2$, $top_k = 4$; ❸ SPARK, set $\alpha = 0.4$, $top_k = 9$; ❹ BGL, set $\alpha = 0.2$, $top_k = 20$.

Evaluation: Training dataset sizes are determined based on established benchmarks [27], [28], [29]. Specifically, DeepLog and LogAnomaly are trained using the first 4,855 normal sequences from HDFS-A and HDFS-B; for the Spark dataset, they utilize the initial 7 normal sequences per task, totaling 203 samples. RobustLog employs a balanced set of first 6,000 normal and 6,000 abnormal sequences for HDFS, and 3,402 samples for Spark (comprising randomly 3,000 normal, 200 abnormal, and

TABLE IV
OPTIMIZATION OF BOOST ALGORITHMS ON HDFS-A

Model		Random Forest	XGBoost	Deep Log	Log Anomaly	Robust Log
Raw	P	**0.9988**	**0.9613**	0.9189	0.9317	**0.9604**
	R	0.4497	0.6207	0.9771	0.9714	0.9951
	F1	0.6202	0.7543	0.9471	0.9512	0.9774
	Tr	66.8ms	52.8ms	3.3m	5.7m	57.1m
	Pr	3.6s	490.2ms	3.5m	3.4m	14.8m
PrefixOpt	P	0.9358	0.9486	0.1381	0.1352	0.9386
	R	0.6009	0.6745	0.9792	0.9529	0.9972
	F1	0.7319	0.7884	0.2419	0.2368	0.9671
	Tr	67.8ms	48.8ms	1.3m	2.1m	33.5m
	Pr	3.4s	437.3ms	40.2s	46.4s	5.4m
LogAssist	P	0.9421	0.9542	0.1449	0.1542	0.9455
	R	0.7231	0.7721	0.9823	0.9677	0.9989
	F1	0.8182	0.8535	0.2525	0.2661	0.9714
	Tr	68.4ms	48.6ms	1.4m	2.2m	34.3m
	Pr	3.2s	441.6ms	44.2s	47.3s	5.1m
LogCleaner	P	0.9523	0.9596	0.8976	0.9578	0.9538
	R	0.9484	0.9528	0.9355	0.9345	0.9669
	F1	0.9503	0.9562	0.9162	0.9460	0.9603
	Tr	**42.2ms**	**27.3ms**	**48.2s**	**1.1m**	**27.1m**
	Pr	**1.7s**	**113.4ms**	**31.1s**	**32.4s**	**3.3m**
LogBoost	P	0.9591	0.9596	**0.9273**	**0.9611**	0.9567
	R	**0.9821**	**0.9761**	**0.9946**	**0.9971**	**0.9996**
	F1	**0.9704**	**0.9678**	**0.9598**	**0.9787**	**0.9777**
	Tr	64.8ms	32.8ms	2.6m	4.4m	33.1m
	Pr	2.4s	139.1ms	39.3s	45.1s	5.2m

• P (Precision), R (Recall), F1 (F1-Score), Tr (Train), Pr (Prediction);
• m (minute), s (second), ms (microsecond).

TABLE V
OPTIMIZATION OF BOOST ALGORITHMS ON HDFS-B

Model		Random Forest	XGBoost	Deep Log	Log Anomaly	Robust Log
Raw	P	**0.9964**	0.9421	**0.9449**	0.9142	0.9438
	R	0.4138	0.4111	0.7531	0.8185	0.9686
	F1	0.5848	0.5723	0.8381	0.8637	0.9561
	Tr	67.9ms	47.7ms	4.8m	7.9m	58.2m
	Pr	3.2s	521.1ms	1.9m	2.7m	14.4m
PrefixOpt	P	0.9862	**0.9607**	0.1251	0.1347	0.0191
	R	0.4318	0.4288	0.8784	0.9248	0.9991
	F1	0.6007	0.5929	0.2191	0.2352	0.0376
	Tr	67.6ms	52.5ms	3.8m	6.2m	33.1m
	Pr	3.2s	487.7ms	1.2m	1.7m	5.2m
LogAssist	P	0.9873	0.9589	0.2311	0.2258	0.0301
	R	0.4625	0.4533	0.8621	0.9124	0.9999
	F1	0.6299	0.6156	0.3645	0.3621	0.0584
	Tr	68.2ms	52.4ms	3.9m	6.3m	35.5m
	Pr	3.3s	491.1ms	1.2m	1.6m	5.4m
LogCleaner	P	0.9473	0.9589	0.9334	0.9268	0.9578
	R	0.9369	0.9533	**0.9413**	0.9244	**0.9899**
	F1	0.9421	0.9561	0.9373	0.9256	0.9736
	Tr	**42.1ms**	**24.3ms**	**2.5m**	**5.1m**	**28.2m**
	Pr	**1.2s**	**108.3ms**	**32.1s**	**1.1m**	**4.5m**
LogBoost	P	0.9441	0.9593	0.9492	**0.9354**	**0.9665**
	R	**0.9582**	**0.9796**	0.9367	**0.9348**	0.9873
	F1	**0.9511**	**0.9693**	**0.9429**	**0.9351**	**0.9768**
	Tr	59.9ms	33.9ms	3.8m	6.2m	35.1m
	Pr	2.4s	150.3ms	47.1s	1.2m	5.1m

the 203 previously selected samples). To ensure performance parity with deep learning models, RandomForest and XGBoost are trained on 500 randomly selected samples, a size validated to yield comparable F1-Scores. All remaining samples are reserved for evaluation.

The evaluation proceeds in two stages: (i) Optimization, 100 iterations of random sampling ($n = 500$ according to RF and XGBoost) are performed to rank and filter the top-k templates by occurrence frequency. This process maintains the original sequence order and applies correspondingly to semantic vectors. (ii) Partitioning, both the original and optimized datasets are split into training and testing sets using the same partition. All experiments are conducted on a Windows 11 workstation equipped with an Intel Core i7-13700K CPU (3.40 GHz), 32 GB RAM, and an NVIDIA RTX 4070 Ti GPU (12GB).

E. RQ1: How Well Can LogBoost Improve the Accuracy and Performance of Anomaly Detection Models?

HDFS-A: As shown in Table IV, it suggests that the removal of templates from log sequences reduces their length, thereby improving the time efficiency of detection models across all optimized datasets. However, only the datasets optimized by LogCleaner and LogBoost exhibit improved F1-Scores relative to the baseline. Although LogCleaner significantly reduces log redundancy, its removal of certain context-sensitive templates renders some abnormal sequences unclassifiable, thereby limiting the potential performance improvement. Meanwhile, PrefixOpt and LogAssist yield a slight improvement for RandomForest and XGBoost, primarily because their frequent itemsets capture templates associated with cyclic patterns. In terms of computational efficiency, LogBoost achieves substantial speedups, but LogCleaner yields more time savings by filtering a larger volume of contextual templates, enabling significantly faster model execution.

HDFS-B: As shown in Table V, the increased number of templates and the complexity of log sequences resulted in a significant decrease in the F1-Score of models. PrefixOpt and LogAssist only yield a slight performance improvement on RandomForest and XGBoost, primarily by pruning redundant decision branches. In contrast, datasets preprocessed by LogCleaner and LogBoost consistently enhance the performance of all models. Notably, while LogCleaner removes more context-sensitive templates, it may lead to a sparsity of log semantics in datasets with high template diversity, thereby capping potential improvements. These results underscore that LogBoost effectively filters out noise while preserving the critical features essential for anomaly detection.

SPARK: The Spark-SDA log dataset is more complex with more log templates and excessively long log sequences. The results on the SPARK dataset are shown in Table VI, LogCleaner and LogBoost improve the F1-Score across all models, while PrefixOpt and LogAssist only improve the XGBoost and RobustLog models. Because Deeplog and LogAnomaly detect anomalies by assessing the probability of the following log template, removing context-dependent log templates can affect their accuracy. Moreover, PrefixOpt, LogAssist, and LogCleaner

TABLE VI
OPTIMIZATION OF BOOST ALGORITHMS ON SPARK

Model		Random Forest	XGBoost	Deep Log	Log Anomaly	Robust Log
Raw	P	1	0.9979	0.5466	0.4917	**1**
	R	0.8317	0.7783	0.7403	0.5848	0.9233
	F1	0.9081	0.8745	0.6289	0.5342	0.9601
	Tr	146.1ms	543.3ms	8.5m	16.9m	27.1m
	Pr	266.2ms	398.2ms	41.5m	69.4m	25.2s
PrefixOpt	P	0.9542	0.9564	0.0562	0.0565	0.9867
	R	0.8106	0.8171	0.9802	0.9534	0.9498
	F1	0.8766	0.8813	0.1063	0.1067	0.9679
	Tr	83.9ms	116.1ms	2.3m	5.3m	27.4m
	Pr	119.1ms	91.9ms	8.3m	15.9m	24.5s
LogAssist	P	0.9634	0.9721	0.0624	0.0612	0.9883
	R	0.8178	0.8311	0.9813	0.9425	0.9499
	F1	0.8847	0.8961	0.1173	0.1149	0.9687
	Tr	85.3ms	131.1ms	2.4m	5.5m	28.3m
	Pr	124.2ms	96.2ms	8.5m	16.2m	25.3s
LogCleaner	P	0.9866	0.9863	0.4337	0.4952	0.9868
	R	0.8263	0.8394	**0.9936**	**0.9845**	0.9516
	F1	0.8994	0.9069	0.6038	0.6589	0.9689
	Tr	**74.6ms**	**96.3ms**	**1.8m**	**3.7m**	**24.2m**
	Pr	**95.2ms**	**81.2ms**	**7.3m**	**12.1m**	**18.6s**
LogBoost	P	**1**	**1**	**0.6755**	**0.5283**	0.9973
	R	**0.8398**	**0.8422**	0.9829	0.9834	**0.9769**
	F1	**0.9129**	**0.9143**	**0.8007**	**0.6873**	**0.9869**
	Tr	114.1ms	306.3ms	5.2m	10.7m	27.1m
	Pr	215.2ms	253.1ms	9.9m	17.6m	24.2s

TABLE VII
OPTIMIZATION F1-SCORE OF FREQUENCY VECTORS

Model		HDFS-A		HDFS-B		Spark	
		RF	XGBoost	RF	XGBoost	RF	XGBoost
Raw	P	0.8926	**0.9646**	0.9893	0.8922	0.9991	1
	R	0.8882	0.7891	0.7721	0.7906	0.8982	0.899
	F1	0.8903	0.8681	0.8673	0.8384	0.9459	0.9468
	Tr	62.1ms	36.1ms	62.1ms	34.3ms	73.5ms	35.2ms
	Pr	2.2s	350.2ms	2.5s	528.2ms	103.6ms	67.4ms
PrefixOpt	P	0.8929	0.9537	0.9767	0.8826	0.9732	0.9413
	R	0.8672	0.7759	0.7236	0.7406	0.8215	0.9192
	F1	0.8798	0.8557	0.8313	0.8053	0.8909	0.9301
	Tr	60.4ms	33.3ms	62.1ms	37.2ms	72.5ms	43.5ms
	Pr	2.1s	350.2ms	2.5s	528.3ms	106.5ms	62.3ms
LogAssist	P	0.9134	0.9612	0.9781	0.9174	0.9834	0.9542
	R	0.8788	0.7834	0.7843	0.8234	0.8633	0.9214
	F1	0.8958	0.8632	0.8705	0.8679	0.9194	0.9375
	Tr	61.4ms	34.4ms	63.3ms	38.2ms	78.1ms	46.3ms
	Pr	2.2s	361.4ms	2.6s	612.4ms	108.4ms	67.2ms
LogCleaner	P	0.9538	0.9489	0.9705	0.9748	0.9842	0.9591
	R	0.9625	0.9739	0.9783	0.9777	0.8989	0.9092
	F1	0.9581	0.9612	0.9744	0.9762	0.9396	0.9335
	Tr	**43.1ms**	**23.1ms**	**44.6ms**	**26.3ms**	**61.1ms**	**26.4ms**
	Pr	**635.2ms**	**238.4ms**	**563.4ms**	**254.6ms**	**87.2ms**	**44.1ms**
LogBoost	P	**0.9791**	0.9595	**0.9989**	**0.9992**	**1**	**1**
	R	**0.9876**	**0.9851**	**0.9879**	**0.9846**	**0.9031**	**0.9135**
	F1	**0.9833**	**0.9721**	**0.9934**	**0.9918**	**0.9491**	**0.9548**
	Tr	60.1ms	27.5ms	59.3ms	32.3ms	77.3ms	31.2ms
	Pr	1.9s	350.3ms	2.3s	520.2ms	107.1ms	60.3ms

remove additional important templates, thereby enabling detection models to operate more efficiently at the expense of a decline in F1-Score. Conversely, LogBoost enhances efficiency without compromising the accuracy.

The following is a summary of the detection models, **(i)** RF and XGB are classification models, and on the LogBoost-optimized dataset they exhibit a comparable F1-Score to that of deep learning models. It demonstrates that LogBoost enhances the distance of various log sequences in the hyperspace, thereby enhancing the classification. In addition, RF and XGB are simple machine-learning models that are much faster to compute than sophisticated models. **(ii)** DeepLog, LogAnomaly and RobustLog are LSTM-based models, reducing the complexity of log sequences and significantly reducing the cost of training and prediction. But DeepLog and LogAnomaly predict the log templates in the sequence one by one, which is more affected by the length of the sequence, especially for SPARK. Moreover, Deeplog and LogAnomaly rely on the hyperparameter *num_classes* in prediction, which is the anomaly threshold based on the probability of log template occurrence. It is difficult to set a suitable threshold facing a large number of log templates and the threshold still requires labeled samples for evaluation. **(iii)** RobustLog is based on semantic features and runs in batches, but it consumes a lot of device memory, and the maximum length of the Spark-SDA needed to be truncated to complete the test in an acceptable time, leading to raw information losing. We validate that a window of 200 is suitable. Because as window size increases, the model accuracy is no longer improved.

Additionally, observing from the training and prediction time shown in HDFS-A, HDFS-B and Spark, the performance of sophisticated models is greatly boosted by LogBoost, and the time is reduced by 20% to 41% in training and 55% to 81% in prediction. RobustLog does not exhibit substantial acceleration on the Spark dataset, as the maximum sequence window size of 200 is smaller than the average lengths of both raw and optimized datasets as shown in Table IX. Moreover, the analysis time of LogBoost for 500 samples is 0.28 seconds for HDFS-A, 0.35 seconds for HDFS-B, and 28.39 seconds for Spark. The analysis time of LogBoost is influenced by the sequence length, and optimization is achieved within a reasonable time. The performance of PrefixOpt, LogAssist, and LogCleaner is significantly influenced by the sequence length, primarily because they rely on frequent itemset search. Consequently, when executed on the Spark dataset, the average sequence length surpasses 3000, rendering them incapable of completing the operation. Therefore, to analyze the Spark dataset with PrefixOpt and LogAssist, we segment its sequence with a window size of 300, which is according to the maximum sequence length of the HDFS dataset.

Frequency Vectors: Since RF and XGB can accept equal-length vectors, such as log frequency vectors in Section II, we evaluate whether template probabilities can achieve high accuracy after LogBoost optimization. We transformed the original and optimized dataset into log frequency vectors for comparison, and the results are shown in Table VII. The performance of PrefixOpt and LogAssist degrades as some pruned high-frequency templates have critical features for anomaly classification. In contrast, LogBoost consistently enhances the F1-score across all datasets and models. While LogCleaner shows improvements on HDFS datasets, its efficacy is constrained elsewhere due to the removal of some high-frequency templates. Notably, HDFS datasets optimized by LogBoost enable RF and XGB to outperform more sophisticated models with a limited training

TABLE VIII
OPTIMIZATION OF BOOST ALGORITHMS ON BGL

Model		Random Forest	XGBoost	Deep Log	Log Anomaly	Robust Log
Raw	P	0.4389	0.5646	0.9012	0.9701	0.9833
	R	0.7853	0.8134	0.9601	0.9412	0.9686
	F1	0.5631	0.6665	0.9313	0.9605	0.9759
	Tr	31.9ms	24.5ms	2.5m	3.9m	29.1m
	Pr	1.6s	271.2ms	1.1m	1.5m	7.2m
PrefixOpt	P	0.4255	0.4782	0.8793	0.9498	0.9586
	R	0.7549	0.7356	0.9399	0.9215	0.9224
	F1	0.5442	0.5796	0.9086	0.9354	0.9402
	Tr	33.8ms	26.4ms	1.9m	3.3m	16.4m
	Pr	1.8s	234.6ms	42.1s	1.1m	2.6m
LogAssist	P	0.5953	0.6784	0.8865	0.9554	0.9733
	R	0.8345	0.8049	0.9486	0.9344	0.9449
	F1	0.6949	0.7363	0.9165	0.9448	0.9589
	Tr	34.1ms	26.5ms	2.2m	2.7m	18.2m
	Pr	1.7s	251.5ms	45.1s	52.1s	2.8m
LogCleaner	P	0.8834	0.9213	0.9243	0.9549	0.9784
	R	0.8776	0.9099	0.9669	**0.9477**	0.9538
	F1	0.8805	0.9156	0.9451	0.9513	0.9659
	Tr	**22.6ms**	**13.6ms**	**1.2m**	**2.6m**	**14.2m**
	Pr	**989.1ms**	**66.1ms**	**18.2s**	**29.1s**	**1.7m**
LogBoost	P	**0.9182**	**0.9294**	**0.9412**	**0.9728**	**0.9856**
	R	**0.8994**	**0.9133**	**0.9781**	0.9411	**0.9746**
	F1	**0.9087**	**0.9212**	**0.9593**	**0.9567**	**0.9801**
	Tr	28.5ms	17.1ms	1.9m	3.2m	17.1m
	Pr	1.3s	78.2ms	23.6s	44.1s	2.5m

TABLE IX
THE STATISTICS OF DATASET WITH/WITHOUT OPTIMIZATION

Dataset	State	HDFS	Spark-SDA	BGL
Log Lines	w/o	10.65 M	15.62 M	4.49 M
	w	8.15 M (↓ 23.5%)	8.36 M (↓ 46.5%)	4.18 M (↓ 6.91%)
Data Size	w/o	1.47 GB	2.22 GB	705 MB
	w	0.98 GB (↓ 33.1%)	1.26 GB (↓ 43.2%)	656 MB (↓ 6.84%)
Max Length	w/o	298	6242	20
	w	68 (↓ 77.2%)	3027 (↓ 51.5%)	20 (↓ -%)
Average Length	w/o	19.4	741.2	20
	w	14.9 (↓ 23.5%)	396.4 (↓ 46.5%)	18.6 (↓ 6.89%)

set of only 500 samples (including a mere 15 to 32 anomalous instances). This robustness under label sparsity underscores the practical utility of LogBoost for real-world applications. Furthermore, the vector dimensionality of the template count (29 for HDFS-A to 158 for SPARK) is substantially lower than the maximum log sequence length (298 to 6,242), significantly reducing the computing time while enabling a higher F1-Score.

BGL: To validate LogBoost on log sequence without IDs, we utilize the BGL dataset for comparison. According to LogAnomaly [28], we set the window size as 20 to obtain a log sequence. The results are shown in the Table VIII. On the raw BGL dataset, the accuracy of RF and XGB is relatively low due to log templates crossing in fixed windows. DeepLog, LogAnomaly, and RobustLog are based on adjacent contexts and are not significantly affected. For optimization, LogCleaner and LogBoost demonstrate performance gains across various models. However, PrefixOpt, LogAssist, and LogCleaner are susceptible to log template overlap, which removes critical high-frequency templates and degrades accuracy for several models. Notably, while LogCleaner implements more aggressive template pruning, resulting in a more pronounced improvement in computational efficiency compared to LogBoost, it also incurs the risk of information loss.

F. RQ2: How Well Can LogBoost Optimize Log Dataset?

The first issue is the maximum length of log sequences in datasets, as it determines the length of the vector padding when training and predicting. The second is the average length distribution of log sequences, which affects the storage and preprocessing. **(i)** HDFS log dataset. As shown in Table IX, LogBoost effectively reduces the length of log sequences by filtering out ineffective log templates. After optimizing, the maximum length is only 68, much less than 298 in the original dataset, reducing the maximum sequence length by more than 77.1%, and most of the sequences are less than 17 (the cumulative frequency is 99.22%). **(ii)** Spark-SDA log dataset. LogBoost also greatly reduces the length of log sequences. The maximum length is only 3027, far from the original dataset with 6250, reducing the maximum sequence length by more than 51.2%, and most of the sequences are distributed less than 1750 (the cumulative frequency is 96.52%). **(iii)** Ultimately, filtering the template resulted in a 23.5% reduction in the number of log records of the HDFS dataset and 46.5% of the Spark dataset.

In contrast, LogBoost exhibits suboptimal performance on the BGL dataset due to constraints in sample selection and template characteristics. Specifically, the abundance of diverse log templates, pronounced variability in window segmentation sequences, and limited template recurrence hinder sequence analysis. Consequently, LogBoost is more suitable for datasets with complete sequences.

G. RQ3: What are the Impacting Factors of LogBoost?

Log parser: It is worth noting that different log parsers may generate distinct templates, which potentially impact the log sequence vector by template ID values. We conduct a comparison of extensively utilized and efficacious log parsers, Spell [39], Drain [46], SwissLog [8] and LogPPT [40] to generate log templates. The results show that LogBoost consistently filters identical log templates of Fig. 7 and overcomes discrepancies in the number of log templates across log parsers. However, the accuracy of log template extraction is a critical factor that can impact the effectiveness of LogBoost, such as some templates may not be properly separated, leading LogBoost to treat them as identical ones.

Sequence length and quantity: The efficiency of LogBoost is mainly affected by the sequence length and the number of training samples. We evaluate LogBoost in different training set sizes, as shown in Table IX, the sequence length of the HDFS dataset is mainly concentrated from 12.5 to 22.5, while Spark is

HDFS-B

ID	Level	Log Template
34	WARN	dfs.DataNode$DataXceiver : <*> : <*> : Got exception while serving <BLK> to <*> :
5	INFO	dfs.FSNamesystem : BLOCK* NameSystem.addStoredBlock : blockMap updated : <*> : <*> is added to <BLK> size <*>
10	INFO	dfs.DataNode$DataXceiver : <*> : <*> Served block <BLK> to <*>
11	INFO	dfs.DataBlockScanner : Verification succeeded for <BLK>

Spark

ID	Level	Log Template	ID	Level	Log Template
62	INFO	MemoryStore : MemoryStore cleared	87	INFO	ShutdownHookManager : Deleting directory <*>
83	INFO	BlockManager : BlockManager stopped	7	INFO	SecurityManager : Changing view acls groups to :
86	INFO	ShutdownHookManager : Shutdown hook called	5	INFO	SecurityManager : Changing view acls to :
30	INFO	CoarseGrainedExecutorBackend : Got assigned task <*>	8	INFO	SecurityManager : Changing modify acls groups to :
31	INFO	Executor : Running task <*> in stage <*> (TID <*>)	2	INFO	SignalUtils : Registering signal handler for TERM

BGL

ID	Level	Log Template	ID	Level	Log Template
301	INFO	<*> double-hummer alignment exceptions	141	INFO	shutdown complete
278	FATAL	guaranteed data cache block touch.. .. .. .. <*>	159	WARN	EndServiceAction is restarting the LinkCards in midplane
443	FATAL	disable store gathering.. .. .. <*>	522	INFO	<*> torus sender <*> retransmission error(s) ...
102	INFO	ddr: Suppressing further CE interrupts	369	INFO	<*> torus processor <*> reception error(s)...
622	FATAL	rts tree/torus link training failed: wanted: <*> got: <*>	171	INFO	<*> <*> directory error(s) (dcr <*> detected ...

Fig. 7. Filtered templates of LogBoost.

TABLE X
COMPARISON OF LOGBOOST WITH/WITHOUT CLASSIFIED SPARK

Metrics	Classified	Random Forest	XGBoost	Deep Log	Log Anomaly	Robust Log
Precision	w/o	0.9133	0.9143	0.5325	0.4412	0.9265
	w	1	1	0.6755	0.5283	0.9973
Recall	w/o	0.7263	0.7258	0.8313	0.7364	0.7836
	w	0.8398	0.8422	0.9829	0.9834	0.9769
F1 Score	w/o	0.8091	0.8092	0.6492	0.5518	0.8491
	w	0.9129	0.9143	0.8007	0.6873	0.9869
Train Time	w/o	141.6ms	366.8ms	5.9m	12.6m	31.4m
	w	114.1ms	306.3ms	5.2m	10.7m	27.1m
Predict Time	w/o	298.7ms	317.1ms	11.2m	20.1m	26.3s
	w	215.2ms	253.1ms	9.9m	17.6m	24.2s

from 150 to 1750. Therefore, the computational time of Spark is significantly higher than that of HDFS due to its excessively long log sequences. For a training sample of 5,000, Spark takes 247 seconds, while HDFS only takes 2.7 seconds. The primary cause of the increased complexity lies in the similarity grouping at the first step, which necessitates traversing all sequences. Consequently, the computational complexity of this process is $O(n)$.

Hyper-parameters: There are two key parameters in LogBoost: α and $topk$. We conduct 100 experiments for each α by randomly selecting training samples and counting the probability of the top-10 filtered templates containing the templates shown in Fig. 7. It validates that when $0.01 < \alpha < 0.4$ and $topk > 2$, LogBoost has more than 90% probability of filtering the optimal templates. Due to α determining the similarity of sequence merging, excessive values may lead to the optimal templates being merged into non-representative sequences, it is also inappropriate to merge sequences with a similarity less than 60% in practical applications.

Sequence classification: As discussed in Section IV-C, LogBoost faces challenges when processing datasets that contain numerous templates with $\Delta SAG_i = 0$. To evaluate this issue, we validate LogBoost on Spark dataset before and after applying sequence classification. As shown in Table X, without classification, some sequences in the training set contain non-overlapping templates, preventing the computation of SAG. This leads to incorrect template deletion and a subsequent decline in model accuracy. Given that LogBoost relies on sequence similarity, we recommend classifying logs from different tasks or components before analysis.

VI. CASE STUDY

To evaluate the impact of LogBoost-filtered log templates on fault localization, we conducted a case study analyzing their effectiveness. Fig. 7 shows the log templates filtered by LogBoost in HDFS-B, SPARK, and BGL datasets. Upon aligning those templates with the original log dataset, we arrived at the following findings. The first log template ID $\{34\}$ with a warning message (WARN) in HDFS-B appears in a large number of normal and abnormal log sequences, which seriously affects the anomaly classification. Moreover, log templates $\{5, 10, 11\}$ with INFO level appeared in contextual subsequences or repeated loops, which have no contribution to classification.

Similarly, The log templates filtered in SPARK are mainly in repeated loops with INFO level. As validated by our fault injection records, the critical fault logs are not included. For the BGL dataset, template 301 appeared 295,764 times, and log templates $\{278, 443, 622\}$ defined as FATAL level appeared frequently but are not marked as faults in the dataset. Therefore, it demonstrates that LogBoost filters ineffective templates, none of which contain critical fault information.

VII. DISCUSSION

The experiments validate the redundant in the raw logs, they are ineffective logs both in normal and abnormal log sequences, and affect the accuracy of anomaly detection, increasing the log sequence length and the cost of storage and analysis. Generally, we need expert knowledge to find these ineffective logs, but LogBoost is a data-driven approach to automatically filter based on raw logs. The results of PrefixOpt indicate that templates derived from template occurrence frequency and frequent pattern mining may encompass crucial templates for sequence classification, and cannot guarantee the impact on anomaly detection performance. In addition, the experiments show the weaknesses of Deep Learning models in the raw Spark-SDA dataset with highly complex log sequences. They require more time for running and parameter adjustment, such as window size and anomaly threshold, which seriously impacts their computation time and accuracy. Conversely, lightweight models, RandomForest and XGBoost, exhibit more feasible and extensive application scenarios but rely on high-quality log datasets. This underscores the imperative to optimize logging and datasets.

A. Threats to Validity

The internal threat to validity mainly lies in the composition of log datasets. Sample selection is critical in data-driven methods, as sample diversity significantly affects outcomes. To ensure robustness, the experiment employs multiple random sampling tests on the whole dataset. In practice, we recommend analyzing as many distinct sequences as possible.

The external threat to validity mainly lies in the hyper-parameters of detection models. We utilize state-of-the-art LSTM-based models that accept log sequence vectors and set

certain optimal parameters based on prior research. Furthermore, the simple machine learning models are set in default parameters, there are still improvements in tuning parameters.

VIII. Related Work

A. Enhancing Log Analysis

Currently, many excellent works use different log enhancement methods to improve log analysis. LogRobust [10] and SwissLog [8] map logs into semantic vectors to improve the robustness and accuracy of detection models. DeepLog [27] and LogAnomaly [28] consider additional features, such as parameter changes and counts of templates to enhance log sequence vectors. Qllog [6] improves the ability of models for unlearned logs by Q-learning. LogKG [47] diagnoses failures based on building knowledge graphs (KG) of logs. LogEncoder [20] utilizes a pre-trained model to obtain more effective semantic vectors of log events. LogLM [3] and SuperLog [48] leverage instruction tuning and domain-specific continual pre-training to bridge the gap between natural language and system logs. LogPPO [49] utilizes Proximal Policy Optimization to align LLM analyses with downstream classifiers, significantly boosting detection confidence in few-shot scenarios. However, the above approaches successfully improve their models, but they do not thoroughly evaluate the influence of each log template on the results.

Furthermore, Logrep [16] investigates the effectiveness of different log representations, revealing that the choice of embedding technique and parsing strategy significantly dictates downstream performance. LTID [50] extracts dependency relation between log statements from source code to remediate the loss of information in the log file. LogAssist [44] applies n-gram modeling to identify common event sequences and hides consecutive events to compress the log events in workflows. LogContrast [1] and LogSentry [12] employ semi-supervised contrastive learning and retrieval-augmented mechanisms to mitigate the impact of log instability and evolving templates.

B. Optimization of Features

Logs consist of structured and unstructured parts, and the keywords in log templates are mainly valid English words or composite tokens in Camel Case [51] format. Since semantic features can improve the robustness of models by keeping the distance between similar words and sentences [8], [9], [10], it is more popular than treating log templates as log events [27], [28]. We investigate dimensionality reduction techniques for semantic features and methods to reduce redundant logs in sequence features through sequence mining.

For semantic features, Stochastic Neighbor Embedding (SNE) [19] and Multi-dimensional Scaling (MDS) [18] map the original features to a lower dimension and retain more effective information for classification. MDS reduces dimensionality by constructing inner product matrices within a lower-dimensional space, ensuring that feature distances match the raw dataset. SNE maps data to probability distributions and reduces the dimensionality based on similarity. In addition, Principle Component Analysis (PCA) [52] is also a common method to reduce the feature dimensionality, but it has limitations on the input features dimension. LogEDL [24] introduces evidential deep learning to quantify the aleatoric and epistemic uncertainty of log features, enhancing the reliability against noisy or out-of-distribution templates. LogCleaner [25] investigates that strategic event pruning can significantly augment model performance by filtering out non-informative or redundant log templates.

For sequence features, the sequence classification field is well established in bioinformatics, web mining or text mining [53]. The method [22] based on sequence coverage to mine robust sequence classification rules. The study [23], [54] uses the Apriori algorithm [55] to mine frequent patterns as features for classification. The work [56] classifies the input action sequences into different categories by mining discriminative subsequences. LogRules [2] proposes a lightweight framework that induces symbolic rules via LLMs, significantly reducing the computational burden of processing sequences with thousands of events while maintaining high interpretability. However, these methods are severely affected by parameters and are difficult to optimize for extremely long log sequences, such as the Spark-SDA log dataset with a more than 3000 average length of sequences.

IX. Conclusion

This paper proposes LogBoost, a data-driven optimization method to filter ineffective log templates to optimize log sequences, it can be easily transferred to the feature engineering step of general classification models as a practical solution. Our experiments show the shortcomings of state-of-the-art methods on complex datasets and validate the effectiveness of LogBoost. With optimized datasets, RandomForest and XGBoost can achieve better performance than sophisticated models given only a small portion of labeled training data, allowing easier few-shot training and prediction. In addition, we construct and release the Spark-SDA dataset, which is helpful in further log optimization research.

Data Availability

The source code, datasets and part of the results of this paper can be found in our replication package [57], [58].

Acknowledgment

The authors greatly appreciate the insightful feedback from the anonymous reviewers.

References

[1] W. Yuan, H. Sun, M. Pang, H. Wang, G. Wu, and Y. Zhang, "LogContrast: Log-based anomaly detection using BERT and contrastive learning," in *Proc. IEEE 23rd Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2024, pp. 2510–2516.

[2] X. Huang, T. Zhang, and W. Zhao, "LogRules: Enhancing log analysis capability of large language models through rules," in *Proc. Findings Assoc. Comput. Linguistics, NAACL 2025*, 2025, pp. 452–470.

[3] Y. Liu et al., "LogLM: From task-based to instruction-based automated log analysis," in *Proc. 2025 IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2025, pp. 401–412.

[4] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–37, 2021.

[5] L. Zhang et al., "LogAttn: Unsupervised log anomaly detection with an autoencoder based attention mechanism," in *Proc. Int. Conf. Knowl. Sci., Eng. Manage.*, 2021, pp. 222–235.

[6] X. Duan, S. Ying, W. Yuan, H. Cheng, and X. Yin, "QLLog: A log anomaly detection method based on Q-learning algorithm," *Inf. Process. Manage.*, vol. 58, no. 3, 2021, Art. no. 102540.

[7] X. Zhang et al., "Onion: Identifying incident-indicating logs for cloud systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, New York, NY, USA, Association for Computing Machinery, 2021, pp. 1253–1263.

[8] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and unified deep learning based log anomaly detection for diverse faults," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, Los Alamitos, CA, USA, IEEE Computer Society, 2020, pp. 92–103.

[9] L. Yang et al., "Semi-supervised log-based anomaly detection via probabilistic label estimation," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.*, Madrid, Spain, IEEE Press, 2021, pp. 1448–1460.

[10] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*New York, NY, USA, Association for Computing Machinery, 2019, pp. 807–817.

[11] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.

[12] L. Wu, X. Wu, and Z. Su, "LogSentry: An LSTM-based framework for real-time vulnerability detection," in *Proc. 2025 Int. Conf. Blockchain Web3. 0 Technol. Innov. Application Exchange 2nd Conf.*, 2025, pp. 165–176.

[13] T. Jia, Y. Li, Y. Yang, G. Huang, and Z. Wu, "Augmenting log-based anomaly detection models to reduce false anomalies with human feedback," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA, Association for Computing Machinery, 2022, pp. 3081–3089.

[14] N. Zhao et al., "An empirical investigation of practical log anomaly detection for online service systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, New York, NY, USA, Association for Computing Machinery, 2021, pp. 1404–1415.

[15] J. Cândido, M. Aniche, and A. van Deursen, "Log-based software monitoring: A systematic mapping study," *PeerJ Comput. Sci.*, vol. 7, 2021, Art. no. e489.

[16] X. Xie, S. Jian, C. Huang, F. Yu, and Y. Deng, "LogRep: Log-based anomaly detection by representing both semantic and numeric information in raw messages," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng.*, 2023, pp. 194–206.

[17] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, and T. Mikolov, "FastText. zip: Compressing text classification models," 2016, *arXiv:1612.03651*.

[18] I. Borg and P. Groenen, "Modern multidimensional scaling: Theory and applications," *J. Educ. Meas.*, vol. 42, no. 3, pp. 277–280, 2005.

[19] G. E. Hinton and S. T. Roweis, "Stochastic neighbor embedding," in *Proc. Adv. Neural Inf. Process. Syst.*, 2003, vol. 15, pp. 833–840.

[20] J. Qi et al., "LogEncoder: Log-based contrastive representation learning for anomaly detection," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1378–1391, Jun. 2023.

[21] S. Huang, Y. Liu, C. Fung, H. Wang, H. Yang, and Z. Luan, "Improving log-based anomaly detection by pre-training hierarchical transformers," *IEEE Trans. Comput.*, vol. 72, no. 9, pp. 2656–2667, Sep. 2023.

[22] E. Egho, D. Gay, M. Boullé, N. Voisine, and F. Clérot, "A parameter-free approach for mining robust sequential classification rules," in *Proc. 2015 IEEE Int. Conf. Data Mining*, 2015, pp. 745–750.

[23] D. Fradkin and F. Mörchen, "Mining sequential patterns for classification," *Knowl. Inf. Syst.*, vol. 45, pp. 731–749, 2015.

[24] Y. Duan et al., "LogEDL: Log anomaly detection via evidential deep learning," *Appl. Sci.*, vol. 14, no. 16, 2024, Art. no. 7055.

[25] L. Zhang, T. Jia, K. Wang, M. Jia, Y. Yang, and Y. Li, "Reducing events to augment log-based anomaly detection models: An empirical study," in *Proc. 18th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2024, pp. 538–548.

[26] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, "Largescale system problem detection by mining console logs," in *Proc. SOSP*, 2009, pp. 117–132.

[27] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Association for Computing Machinery, 2017, pp. 1285–1298.

[28] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Macao, China, AAAI Press, 2019, no. 7, pp. 4739–4745.

[29] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, New York, NY, USA, Association for Computing Machinery, 2019, pp. 807–817.

[30] I. Mavridis and H. Karatza, "Performance evaluation of cloud-based log file analysis with apache hadoop and apache spark," *J. Syst. Softw.*, vol. 125, pp. 133–151, 2017.

[31] S. He et al., "An empirical study of log analysis at Microsoft," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2022, pp. 1465–1476.

[32] N. Busany and S. Maoz, "Behavioral log analysis with statistical guarantees," in *Proc. 38th Int. Conf. Softw. Eng.*, 2016, pp. 877–887.

[33] J. Pei et al., "PrefixSpan,: Mining sequential patterns efficiently by prefix-projected pattern growth," in *Proc. 17th Int. Conf. Data Eng.*, 2001, pp. 215–224.

[34] S. Salvador and P. Chan, "Toward accurate dynamic time warping in linear time and space," *Intell. Data Anal.*, vol. 11, no. 5, pp. 561–580, 2007.

[35] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2007, pp. 575–584.

[36] B. Yu et al., "Deep learning or classical machine learning? An empirical study on log-based anomaly detection," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2023, pp. 392–404.

[37] E. Paxinou, D. Kalles, C. T. Panagiotakopoulos, and V. S. Verykios, "Analyzing sequence data with Markov chain models in scientific experiments," *SN Comput. Sci.*, vol. 2, no. 5, pp. 1–14, 2021.

[38] Chaosblade. io, "Chaosblade: An easy-to-use and powerful chaos engineering experiment toolkit," 2025. [Online]. Available: https://github.com/chaosblade-io/chaosblade

[39] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *Proc. IEEE 16th Int. Conf. Data Mining*, Los Alamitos, CA, USA, IEEE Computer Society, 2016, pp. 859–864.

[40] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 2438–2449.

[41] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[42] A. Liaw et al., "Classification and regression by randomforest," *R News*, vol. 2, no. 3, pp. 18–22, 2002.

[43] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, Association for Computing Machinery, 2016, pp. 785–794.

[44] S. Locke, H. Li, T.-H. P. Chen, W. Shang, and W. Liu, "LogAssist: Assisting log analysis through log summarization," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3227–3241, Sep. 2022.

[45] M. Martini, D. Schuster, and W. M. van der Aalst, "Mining frequent infix patterns from concurrency-aware process execution variants," *Proc. VLDB Endowment*, vol. 16, no. 10, pp. 2666–2678, 2023.

[46] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. 2017 IEEE Int. Conf. Web Serv.*, Los Alamitos, CA, USA, IEEE Computer Society, 2017, pp. 33–40.

[47] Y. Sui et al., "LogKG: Log failure diagnosis through knowledge graph," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3493–3507, Sep./Oct. 2023.

[48] G. Wei and L. Xiao, "Super log-concavity of the first eigenfunctions for horo-convex domains in hyperbolic space," 2025, *arXiv:2510.13072*.

[49] Z. Wang, J. Dong, and C. Yang, "LogPPO: A log-based anomaly detector aided with proximal policy optimization algorithms," *Smart Cities*, vol. 9, no. 1, p. 5, 2026.

[50] J. Zhao, Y. Tang, S. Sunil, and W. Shang, "Studying and complementing the use of identifiers in logs," in *Proc. 2023 IEEE Int. Conf. Softw. Anal., Evol. Reengineering*, 2023, pp. 97–107.

[51] B. Dit, L. Guerrouj, D. Poshyvanyk, and G. Antoniol, "Can better identifier splitting techniques help feature location?," in *Proc. IEEE Int. Conf. Prog. Comprehension*, 2011, pp. 11–20.

[52] I. T. Jolliffe and J. Cadima, "Principal component analysis," *Wiley Interdiscipl. Rev., Comput. Statist.*, vol. 4, no. 2, pp. 124–167, 2012.

[53] Z. Xing, J. Pei, and E. Keogh, "A brief survey on sequence classification," *ACM Sigkdd Explorations Newslett.*, vol. 12, no. 1, pp. 40–48, 2010.
[54] T. P. Exarchos, M. G. Tsipouras, C. Papaloukas, and D. I. Fotiadis, "A two-stage methodology for sequence classification based on sequential pattern mining and optimization," *Data Knowl. Eng.*, vol. 66, no. 3, pp. 467–487, 2008.
[55] M. Hegland, "The apriori algorithm–a tutorial," in *Mathematics and Computation in Imaging Science and Information Processing*. Singapore: World Scientific, 2007, pp. 209–262.
[56] S. Nowozin, G. Bakir, and K. Tsuda, "Discriminative subsequence mining for action classification," in *Proc. IEEE 11th Int. Conf. Comput. Vis.*, 2007, pp. 1–8.
[57] limin, "Spark-sda dataset," 2025. [Online]. Available: https://github.com/limin5120/Spark-SDA
[58] limin, "Logboost," 2025. [Online]. Available: https://github.com/limin5120/LogBoost#

Min Li received the master's degree from the Graduate School of China Academy of Engineering Physics in 2022. He is currently working toward the PhD degree with the School of Systems Science and Engineering, Sun Yat-sen University, Guangzhou, China. His research interests include log analysis and LLM-based system operations.

Xiaoyun Li received the BE degree in 2019 from Sun Yat-sen University, where she is currently working toward the PhD degree with the School of Computer Science and Engineering. Her research interests include log analysis and AI-driven operations.

Pengfei Chen received the PhD degree from the Department of Computer Science, Xi'an Jiaotong University in 2016. He is currently an associate professor with the School of Computer Science and Engineering, Sun Yat-sen University. He is also a PhD advisor. He has authored or coauthored more than 50 papers in some international conferences including IEEE INFOCOM, WWW, ACM/IEEE CCGRID, ICSOC, IEEE ICWS, IEEE ICPADS and journals including*IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on Reliability*, *IEEE Transactions on Services Computing* , *IEEE Transactions on Emerging Topics in Computing*, and *IEEE Transactions on Cloud Computing*. His research interests include distributed systems, AIOps, cloud computing, microservice and blockchain. Especially, he has strong skills in cloud computing. He is a program committee member of multiple conferences and reviewer of some internal journals such as *IEEE Transactions on Cybernetics*, *Information Science*, and *Neurocomputing*.

Yuanhao Lai received the PhD degree in statistics from Western University, London, ON, Canada, in 2020. He is currently a senior researcher with Huawei Technologies Company Ltd. His research interests include statistical learning, statistical computing, and cloud computing.

Zibin Zheng (Fellow, IEEE) received the PhD degree in computer science and engineering from the Chinese University of Hong Kong. He is currently a professor and deputy dean of the School of Software Engineering, Sun Yat-sen University, Guangzhou, China. He authored or coauthored more than 200 international journal and conference papers, including one ESI hot paper and six ESI highly cited papers. His research interests include blockchain, software engineering, and services computing. He is a fellow of IET. He was the recipient of several awards, including the Top 50 Influential Papers in Blockchain of 2018, ACM SIGSOFT Distinguished Paper Award at ICSE2010, and Best Student Paper Award at ICWS2010.